Chapter 1 Study Notes: Foundations & Evolution of Deep Learning

1. Resolving Core Confusions - Comparison Reference

Common Industry Misunderstandings

Concept A	Concept B	Key Distinctions
Machine Learning	Deep Learning	Machine Learning is a broad discipline where a program is defined by learning from data rather than being entirely hard-coded. Deep Learning is a specialized subset of machine learning that specifically uses neural networks containing multiple layers.
Architecture	Model	Architecture is the general template or mathematical function form that we fit. A model is the specific combination of that architecture with a particular, finalized set of parameters.
Loss	Metric	Loss is a measure of performance used to guide training and update parameters automatically via optimization. A metric is a measurment of model quality calculated on the validation set strictly for human consuption and interpretation.
Classification	Regression	Classification models attempt to predict a discrite category or class from distinct possibilties (e.g. "dog" or "cat"). Regression models attempt to predict one or more continous numeric quantities (e.g. a temperature or location)

Chapter 7 continued

Concept A	Concept B	Key Distinctions
Training Set	Validation Set	Training Set is the data used to directly fit and update the model parameters. Validation Set is held out from training entirely and used solely to measure the model's accuracy and track generalize
Validation Set	Test Set	Validation Set is used by the modeler to evaluate multiple architectures and tune hyperparameters. Test set is highly reserved, totally hidden cut of data that is locked away even from the modeler to perform final, unbiased evaluation.

2. Core Topics

Topic A: Arthur Samuel's Terminology & Modern Translation

In 1949, IBM researcher Arthur Samuel realized that instead of writing detailed, step-by-step instructions for a computer to solve a task, we could show it examples of the problem and let it figure out the solution itself. To make this automatic, he designed a system that adjusts it's internal variables to maximize actual performance.

To connect Samuel's classic checkers-players model to modern deep learning, we translate his terms directly:

- Architecture: The functional form or underlying mathematical template of the model. In Samuel's checkers, this represents the general playbook of strategies.
- Parameters: The variables inside the model that define how it actually operates and what task it can do. These are what Samuel refered to as "weight assignments" or "weights."

chp 1 continued

- Independent Variable: The raw input data, not including the lables (such as image pixels or board states)

- Predictions: The calculated outputs of the model based on the independent variables.

- Targets / Dependent Variable / Labels: The correct answers we are trying to predict (e.g. image is a dog or cat)

- Loss: the mathamatical smooth measure used to drive a training loop. It scores how bad prediction is, allowing the system to update weights automatically.

- Update: The automatic mechanism that alters parameter values to improve performance based on the loss.

Once a model is fully trained, the parameters are locked. At this point, the trained model can be treated exactly like a traditional computer program.

Topic B: Universal Approximation & Stochastic Gradient Descent (SGD)

- A neural network is simply a specific kind of machine learning model. It is represented as a mathematical function that is extremly flexible. The Universal Approximation Theorem is the mathematical proof that this function is flexible enough to solve any continuous problem to any level of accuracy, in theory, simply by finding the correct weight parameters.

- To automatically find those optimal weights, we use an optimization algorithm called Stochastic Gradient Descent (SGD). In practice neither neural networks or SGD are mathamatically complex. The rely heavly on massive volumes of simple addition and multiplication to calculate predictions and automatically slide the weights "downward" toward the lowest possible loss.

Topic C: Overfitting and the Data Exposure Hierarchy

- The single most challenging issue when training any machine learning model is overfitting. If you train a model too long on any dataset it will eventually memorize the specific features of that training data rather than discovering the general, underlying patterns. "It acts like a student memorizing practice test answers and failing on the real exam."

- Dividing Data: to keep ourselves honest and ensure our model actually generalizes to new, unseen data, we divide our dataset into a clear hierarchy based on how much data we expose:

[Total Dataset]
- Training Set (Fully Exposed: SGD directly fits weight param)
- Validation Set (Semi-Exposed: Modeler adjusts hyperparameters)
- Test Set (Totally Hidden: Locked away until final eval)

1. **Training Set:** This is the data the model looks at to directly learn. The optimization algorithm directly looks at this data to update the model's weight parameters.

2. **Validation Set:** This data is held out from training entirely. We use it to measure accuracy on unseen data and to help us make choices about our training setup. The higher-level settings we adjust (such as learning rate or network structure) are called **hyperparameters** - meaning parameters that govern how weight parameters are learned.

3. Test Set: Because we are constantly evaluating the validation set to tweak our hyperparameters, we are in danger of indirectly overfitting the validation set through our own human trial and error. To prevent this self deception we reserve a test set. It must be kept completly secret from both the training system and our own design decisions, evaluated exactly once at the very end of the project to check real world accuracy.

chapter 7 notes continued
(part B)

Designing Representative Validation Sets: Standard random splits often fail in production:

• Time Series: A random split lets the model look at past and future dates to cheat. To simulate real-world usage, we must choose a continuous block of the latest dates as validation set.

-- 04 Under the hood of the image classifier --

We previously built an image classifier that recognized cats and dogs. This breaks down the fastai code used to build that.

- Data Preparation -

• The fastai library uses classes like ImageDataLoaders to handle the structure of different datasets.

• Functions extract labels programmatically from file paths or filenames.

• Transform: code applied automatically during training to format data.

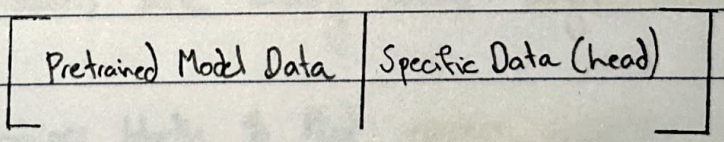
%item-tfms> apply to each item individually, like resizing to a 224-pixel square.

%batch-tfms> apply to batches simultaneously using GPU for speed.

- Convolution Neural Networks and Pretrained Models -

<vision_learner> creates a convolution neural network (CNN), the state-of-the-art approach for computer vision.

- Pretrained models (like ResNet34) come pretrained on millions of images.
  - Saves time and data because the models already detect edges, gradient, and colors.
  - **Transfer Learning**, using a pretrained model for a task different from it's original training.
    a. fastai library does this by removing the pretrained model's final layer and replaces it with a randomized layer specific to the new dataset, called the **head**.
    b. This



- Fitting the model -

- **Epoch**: One complete passthrough the input data
- **Fine-tuning**: Updating a pretrained model's parameters for a new task.
- **Fit**: General process of training a model to learn patterns from a dataset.
- <fine_tune> executes in two steps
  - Fits just the newly added head for one epoch.
  - Fits the entire model for requested epochs, updating later layers faster than earlier layers.

chpter 1 continued

- Metrics vs loss -

• Metric: Function measuring model prediction quality using the validation set, designed for human comprehension.

• Loss: Performance measure chosen to drive training via stochastic gradient descent.

° metric is for humans; loss is for machines (loss's output is not intuitive or meaningful)

--05 What the Network Learns and Creative Transformations--

How CNN recognize patterns and how non-image problems can be solved using image recognizers.

= Visualizing CNN layers - (pge 34-36)

•layer 1 discovers baric building blocks: diagnal lines, horizontal edges, color gradients

•layer 2 combines blocks to find: corners, repeating lines, circles

•layer 3 and beyond identify high-level semantic components like: car wheels, text, flower petals.

• Fine-tuning adapts highly specialized later layers to focus on specific tasks.

-Image recognizers for non-image tasks-

• If the human eye recognizes categories from an image representation, a deep learning model can too.

-- 06 Beyond Image Classification --

Code Patterns used for segmentation, text analysis, tabular data, and recommendation systems.

- Segmentation -

- Segmentation: Creating a model that recognizes the content of every individual pixel in an image.
  - Autonomous vehicles use segmentation to precisely locate pedestrians.

- Natural Language Processing (NLP) -

- NLP models generate text, translate languages, analyze sentiment, and label words.
- TextDataloaders and <text_classifier_learner> process and classify documents like movie reviews.

- Tabular Data -

- Tabular: Data in table form, such as spreadsheets, databases, or CSV files.
- Categorical columns contain discrete sets of choices, like occupation.
- Continuous columns contain numeric quantities, like age
- Tabular models train from scratch using <fit_one_cycle> because pretrained models are generally unavailable.

Chapter 7 continued

- Collaborative filtering -

• Recommendation systems predict products or movies users might like based on viewing habits.

• `y_range`, the target constraint. Tells the model the exact range of continuous target numbers to predict, stabilizing training.

-- 07 Intellectually Honest Evaluations --

Lesson covers splitting data to evaluate models honestly without accidentally cheating.

- Validation Set and Test Set -

• **Training Set:** Data the model sees and learns from during fitting.

• **Validation Set:** Data held out from training, used only to measure model quality.

◦ **Overfitting:** Training a model on data to a point that it remembers specific features of training data and performs poorly on new data.

◦ **Hyperparameters:** Higher-level choices governing weight parameters, like architecture and learning rates.

◦ **Test set:** Third level of data held back even from the modeler.

a. Prevents human overfitting of validation data through hyperparameter trial and error.

b. Only used to evaluate the model at the very end.